

XAudio2Player Sample 4 – Technical Architecture (One Page)

Focus: Media Foundation streaming decode + XAudio2 multi-buffer playback + worker-thread command/state machine. This page highlights the runtime flow, buffer lifecycle, and timing math used for accurate UI position reporting.

GUI (VCL thread)	Engine Worker Thread	Media Foundation	XAudio2 Audio Thread
- Buttons: Load/Play/Pause/Stop/WaitPlay/Seek - Shows progress + time - Receives callbacks via TThreadQueue	- WaitPlay/Seek event - Handle cmdKind - Read/Refill buffers - PollStateAndNotify	- IMFSourceReader - SetCurrentPosition (seek) - ReadSample → PCM	- IXAudio2SourceVoice - SubmitSourceBuffer - OnBufferEnd → signal refill

Execution / Threading Flow

```
RUNTIME FLOW (simplified sequence)
-----
GUI: LoadAndInitializeAsync(file, AutoPlay=False)
-> RequestCommand(cmdLoad(file, AutoPlay))
Worker: cmdLoad
- Stop/Flush old voice buffers (if any)
- SetupSourceReader(file) [MF attributes + float PCM output]
- InitializeXAudio2(False) [Create voice + effects]
- PrimeStreaming() [fills STREAM_BUFFER_COUNT buffers]
- pvRenderStatus := rsInitialized
- Queue(OnAudioReadyEvent)

GUI: Play
-> RequestCommand(cmdPlay)
Worker: cmdPlay
- (optional) XAudio2.StartEngine if not using STOP_ENGINE_WHEN_IDLE
- SourceVoice.Start
- pvRenderStatus := rsPlaying
- Queue(OnAudioPlayingEvent)

XAudio2 thread: OnBufferEnd(bufferContext)
-> FBufEndEvent.SetEvent / InterlockedIncrement(FNeedRefill)

Worker loop (each iteration):
1) handle command if FCmdEvent signaled
2) if rsPlaying: PollStateAndNotify() [position/time UI]
3) if NeedRefill>0: SubmitNextStreamBuffer() until caught up
```

Buffer Lifecycle

```
BUFFER LIFECYCLE (multi-buffer streaming)
-----
PrimeStreaming():
for i in 0..STREAM_BUFFER_COUNT-1:
  ReadPcmInto(StreamBuf[i].Data, StreamBuf[i].Cap, Wrote)
  xa.Flags := (Wrote==0 || EOF) ? XAUDIO2_END_OF_STREAM : 0
  xa.pAudioData := StreamBuf[i].Data
  xa.AudioBytes := Wrote
  xa.pContext := @StreamBuf[i] (or index)
  SubmitSourceBuffer(xa)

Refill:
OnBufferEnd -> signal worker -> SubmitNextStreamBuffer(IsLast)
- reuses ring buffer slot FNextSubmit
- reads next PCM chunk
- submits next XAUDIO2_BUFFER
- when EOF: set XAUDIO2_END_OF_STREAM and set FReachedEof := True
- when buffers run dry: pvRenderStatus can transition to rsEndOfStream
```

Seek / Replay Semantics

SEEK / REPLAY (streaming model)

GotoNewPosition(SamplePos):

- SourceVoice.Stop / FlushSourceBuffers / Discontinuity
 - MF seek: FReader.SetCurrentPosition(hns = SamplePos * 10⁷ / SamplesPerSec)
 - reset streaming flags: FReachedEof:=False; NeedRefill:=0
 - pvSeekBaseSamples := SamplePos
 - PrimeStreaming() (submits fresh buffers at new MF position)
 - IMPORTANT baseline (cumulative SamplesPlayed):
- pvSourceVoice.GetState(st,0)
pvBufferPrevPlayed := st.SamplesPlayed
- if wasPlaying then SourceVoice.Start

Replay button:

- GotoNewPosition(0)
- Play

Timing / Position Tracking

TIMING / UI POSITION MATH (avoids "resume-at-old-position" bug)

XAUDIO2_VOICE_STATE.SamplesPlayed is cumulative for the life of the SourceVoice.
Therefore compute delta vs. baseline:

playedAbs := st.SamplesPlayed
playedDelta := max(0, playedAbs - pvBufferPrevPlayed)
absSamples := pvSeekBaseSamples + playedDelta
timeHns := absSamples * 10⁷ / pvSamplesPerSecond

Engine publishes:

FXaudio2EventData.Position = pvSeekBaseSamples
FXaudio2EventData.SamplesProcessed = playedDelta
FXaudio2EventData.TimePlayed = timeHns